

INSTALLATION GUIDE

FlowAccel / JotFlow

Complete step-by-step setup on a fresh Windows VM

Workflow Management Dashboard • React + Express + PostgreSQL on IIS
Document version 1.0 • Generated 19 May 2026

Contents

1. Overview & Architecture
2. VM Requirements (what kind of machine you need)
3. Part A — Install required software (Node.js, PostgreSQL, IIS, PM2)
4. Part B — Copy the project files onto the VM
5. Part C — Install project dependencies
6. Part D — Create and migrate the database
7. Part E — Configure the backend (`.env` file)
8. Part F — Build the frontend
9. Part G — Deploy the site to IIS
10. Part H — Start the backend with PM2
11. Part I — Create the first login user
12. Part J — Verify everything works
13. Part K — Optional: auto-start on boot, HTTPS, firewall / LAN access
14. Troubleshooting
15. Quick Reference

Who this guide is for: anyone setting up FlowAccel on a brand-new Windows machine or virtual machine. Every command is written out in full — copy and paste them exactly. Estimated time: **45–90 minutes** (most of it is downloads and installers).

1. Overview & Architecture

FlowAccel (internally called **JotFlow**) is a workflow / approval dashboard. It has two parts that run together:

Part	What it is	Where it runs
Frontend	The website you see in the browser (React app, built into static HTML/JS/CSS files in a <code>dist</code> folder).	Served by IIS on port 80 .
Backend	The API server (Node.js + Express) that handles login, data, and talks to the database.	A Node process kept alive by PM2 on port 3000 .
Database	Stores users, submissions, approvals, etc.	PostgreSQL on port 5432 .

How a request flows

```
Browser → http://<vm>/          → IIS serves the React app (static files)
Browser → http://<vm>/api/...    → IIS reverse-proxies → Node backend :3000
Browser → http://<vm>/socket.io/.. → IIS reverse-proxies → Node backend :3000
Browser → http://<vm>/uploads/... → IIS reverse-proxies → Node backend :3000
                                   Node backend → PostgreSQL :5432
```

IIS does the reverse-proxying using two add-on modules: **URL Rewrite** and **Application Request Routing (ARR)**. The rules live in the `web.config` file that ships with the project.

Why this design? The browser only ever talks to one address (port 80). IIS quietly forwards the `/api` calls to the Node backend behind the scenes, so there are no cross-origin / CORS headaches and only one port needs to be open to users.

2. VM Requirements

Item	Minimum	Recommended
Operating system	Windows 10/11 Pro or Windows Server 2019	Windows Server 2022
CPU	2 vCPU	4 vCPU
RAM	4 GB	8 GB
Disk	20 GB free	40 GB free
Network	Internet access (for downloads)	Static IP for the VM
Access	An account with Administrator rights	—

You must be an Administrator on the VM. Installing IIS, creating websites, and copying files into `C:\inetpub` all require admin rights. When this guide says "open PowerShell as Administrator", right-click the Start menu → `Windows PowerShell (Admin)` / `Terminal (Admin)`.

Software you will install (Part A)

Software	Version	Purpose
Node.js	20 or 22 LTS	Runs the backend and builds the frontend
PostgreSQL	15 or newer	Database
IIS	Built into Windows	Web server for the frontend + reverse proxy
URL Rewrite + ARR	Latest	IIS add-ons that enable the <code>/api</code> proxy
PM2	Latest (via npm)	Keeps the Node backend running
Git (optional)	Latest	Only if you clone the code instead of copying a folder

Part A — Install Required Software

A1 Install Node.js

1. Open <https://nodejs.org> in the VM's browser.
2. Download the **LTS** installer for Windows (`.msi` , 64-bit).
3. Run it. Accept all defaults — **keep "Add to PATH" ticked**. Click through to Finish.
4. Verify: open a **new** PowerShell window and run:

```
node -v
npm -v
```

You should see something like `v22.x.x` and `10.x.x` . If you get "not recognized", close and reopen PowerShell, or restart the VM.

A2 Install PostgreSQL

1. Open <https://www.postgresql.org/download/windows/> → "Download the installer".
2. Download PostgreSQL **15** (or newer) and run the installer.
3. During setup:
 - **Installation directory** — leave default (`C:\Program Files\PostgreSQL\15`).
 - **Components** — keep "PostgreSQL Server" and "Command Line Tools" ticked. pgAdmin is optional.
 - **Password** — set a password for the `postgres` superuser. **Write it down** — you need it later. This guide assumes `postgres` .
 - **Port** — leave `5432` .
 - **Locale** — default.
4. Finish. You can skip "Stack Builder" at the end.

Add psql to PATH (handy): so you can run `psql` from any folder, add `C:\Program Files\PostgreSQL\15\bin` to the system `Path` environment variable. Otherwise, use the full path to `psql.exe` as shown in Part D.

A3 Install IIS + URL Rewrite + ARR

A3.1 — Turn on IIS

Open **PowerShell as Administrator** and run the one command for your OS:

Windows 10 / 11:

```
Enable-WindowsOptionalFeature -Online -All -FeatureName `
  IIS-WebServerRole, IIS-WebServer, IIS-CommonHttpFeatures, `
  IIS-StaticContent, IIS-DefaultDocument, IIS-HttpErrors, `
  IIS-HttpLogging, IIS-RequestFiltering, IIS-Security, `
  IIS-WebSockets, IIS-ManagementConsole -NoRestart
```

Windows Server:

```
Install-WindowsFeature -Name Web-Server, Web-WebSockets, `
  Web-Mgmt-Console, Web-Static-Content, Web-Default-Doc, `
  Web-Http-Errors, Web-Http-Logging, Web-Filtering -IncludeManagementTools
```

When it finishes, confirm IIS is running:

```
Get-Service W3SVC
```

Status should be `Running` . You can also browse to `http://localhost/` — the default IIS welcome page should appear.

A3.2 — Install the URL Rewrite module

1. Download from <https://www.iis.net/downloads/microsoft/url-rewrite> (file: `rewrite_amd64_en-US.msi`).
2. Run the installer → Next → Finish.

A3.3 — Install Application Request Routing (ARR)

1. Download from <https://www.iis.net/downloads/microsoft/application-request-routing> (file: `ARRv3_0.exe` / "ARR 3.0").
2. Run the installer → Next → Finish.

Both URL Rewrite and ARR are mandatory. Without them the `/api` calls cannot reach the backend and the app will load but fail to log in.
Install URL Rewrite **before** ARR.

A4 Install PM2

In **PowerShell as Administrator**:

```
npm install -g pm2
```

Verify:

```
pm2 -v
```

Part B — Copy the Project Files onto the VM

Put the whole project folder somewhere on the VM. This guide uses `C:\Apps\flowaccel` as the example location — use whatever you like, but remember it.

Option 1 — Copy a folder

If you have the project as a folder (USB drive, network share, zip), copy it so the final structure looks like:

```
C:\Apps\flowaccel\  
├─ backend\          (Express API source)  
├─ frontend\         (React app source)  
├─ web.config        (IIS rules)  
├─ deploy-to-iis.ps1 (deployment script)  
└─ package.json
```

Option 2 — Clone with Git

If the code is in a Git repository, install Git from <https://git-scm.com/download/win>, then:

```
mkdir C:\Apps  
cd C:\Apps  
git clone <your-repository-url> flowaccel
```

Throughout the rest of this guide, `C:\Apps\flowaccel` means "wherever you put the project". If you used a different path, substitute it everywhere.

Part C — Install Project Dependencies

The project has two parts (backend and frontend), each needs its own `npm install`. In PowerShell:

```
cd C:\Apps\flowaccel\backend
npm install

cd C:\Apps\flowaccel\frontend
npm install
```

Each command downloads packages into a `node_modules` folder. This can take a few minutes. Warnings are normal; **errors** in red that stop the command are not — re-run if the network dropped.

Part D — Create and Migrate the Database

D1 Create the database

Run this in PowerShell. It connects as the `postgres` user and creates an empty database called `jotflow`. You will be prompted for the `postgres` password you set in Part A2.

```
& "C:\Program Files\PostgreSQL\15\bin\createdb.exe" -U postgres -h localhost jotflow
```

No output means success. If you see "database already exists", that's fine.

D2 Apply the schema (migration)

The project ships a migration script that creates all the tables. First make sure the `.env` file exists (Part E) — actually, do Part E now, then come back. Or set the database URL inline just for this step:

```
cd C:\Apps\flowaccel\backend
node db/migrate.js
```

Expected output:

```
[migrate] Connected to PostgreSQL
[migrate] Schema applied successfully
```

`migrate.js` reads the database connection from `backend\.env` (`DATABASE_URL`). So you must create the `.env` file (Part E) **before** running the migration. The ordering is: Part E first, then D2.

The migration creates all tables and automatically seeds one default organization, so you are ready to create users immediately afterwards.

Part E — Configure the Backend (`.env` file)

The backend reads its settings from a file called `.env` in the `backend` folder. This file does **not** exist by default — you create it.

E1 Generate a session secret

Run this once and copy the long string it prints:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

E2 Create `backend\.env`

Create a new file `C:\Apps\flowaccel\backend\.env` with this content (use Notepad — make sure it saves as `.env`, not `.env.txt`):

```
# — PostgreSQL —
DATABASE_URL=postgresql://postgres:YOUR_PG_PASSWORD@localhost:5432/jotflow

# — Session (paste the string from step E1) —
SESSION_SECRET=PASTE_THE_LONG_RANDOM_STRING_HERE

# — Server —
PORT=3000
NODE_ENV=development

# — CORS — the address(es) users will open in the browser —
ALLOWED_ORIGIN=http://localhost,http://YOUR_VM_IP

# — JotForm API (only needed for live JotForm sync; leave blank otherwise) —
JOTFORM_API_KEY=
JOTFORM_TEAM_ID=
JOTFORM_BASE=https://eforms.mediaoffice.ae/API
JOTFORM_HOST=https://eforms.mediaoffice.ae
JOTFORM_WEBHOOK_SECRET=

# — Microsoft SSO (optional — leave blank to disable) —
MICROSOFT_CLIENT_ID=
MICROSOFT_CLIENT_SECRET=
MICROSOFT_TENANT_ID=
MICROSOFT_REDIRECT_URI=
```

What to change

Setting	Set it to
<code>YOUR_PG_PASSWORD</code>	The <code>postgres</code> password from Part A2.
<code>SESSION_SECRET</code>	The random string from step E1.
<code>YOUR_VM_IP</code>	The VM's IP address (run <code>ipconfig</code> to find it, e.g. <code>192.168.1.50</code>). If users will only use the VM locally, you can leave just <code>http://localhost</code> .
<code>JOTFORM_*</code>	Fill in only if you use live JotForm integration. The app runs fine with them blank.

`NODE_ENV` controls how login works:

- `development` — any account in the database can log in. **Use this** unless the VM is inside the GDMO network.
- `production` — every login is first checked against the JotForm workspace (`eforms.mediaoffice.ae`). If the VM cannot reach that host, **all logins fail**. Only use `production` if the VM has network access to JotForm Enterprise and you have a valid `JOTFORM_API_KEY`.

After creating `.env`, go back and run the database migration (**Part D, step D2**) if you have not already.

Part F — Build the Frontend

This compiles the React app into static files inside a `dist` folder at the project root.

```
cd C:\Apps\flowaccel\frontend
npm run build
```

It finishes with a line like `✓ built in 20s`. Afterwards a `C:\Apps\flowaccel\dist` folder exists containing `index.html`, an `assets` folder, and others.

Re-run `npm run build` any time you change the frontend code, then re-run the deploy script (Part G) to publish the new build.

Part G — Deploy the Site to IIS

The project includes a script — `deploy-to-iis.ps1` — that does the whole IIS setup for you:

- Copies the built frontend (`dist`) into `C:\inetpub\flowaccel`
- Copies `web.config` (the reverse-proxy rules)
- Enables the ARR reverse proxy at the server level
- Creates an IIS application pool and website on **port 80**
- Sets folder permissions

G1 Run the deploy script as Administrator

Open **PowerShell as Administrator** and run:

```
cd C:\Apps\flowaccel
PowerShell -ExecutionPolicy Bypass -File .\deploy-to-iis.ps1
```

The script prints its progress and finishes with `=== DONE — open http://localhost/ ===`. It also writes a log to `%TEMP%\flowaccel-iis-deploy.log`.

The script figures out the project location automatically (it uses its own folder). You do **not** need to edit any paths inside it — just keep `deploy-to-iis.ps1` in the project root.

If port 80 is already used by another site, the script stops the default IIS site to free it. If you need FlowAccel on a different port, edit the binding line in `deploy-to-iis.ps1` (`/bindings:'http/*:80:'` → e.g. `'http/*:8080:'`) before running it.

G2 What gets created

IIS object	Value
Website name	<code>FlowAccel</code>
Application pool	<code>FlowAccel</code> (No Managed Code)
Physical path	<code>C:\inetpub\flowaccel</code>
Binding	<code>http</code> on port <code>80</code>

Part H — Start the Backend with PM2

IIS is now serving the frontend, but the API backend still needs to be running. PM2 starts it and keeps it alive.

```
cd C:\Apps\flowaccel\backend
pm2 start ecosystem.config.js
pm2 save
```

Check it is online:

```
pm2 list
```

You should see `jotflow-backend` with status `online` . Useful commands:

Command	Does
<code>pm2 logs jotflow-backend</code>	View live backend logs
<code>pm2 restart jotflow-backend</code>	Restart the backend
<code>pm2 stop jotflow-backend</code>	Stop the backend
<code>pm2 list</code>	Show all processes and status

The backend runs from the project folder (`C:\Apps\flowaccel\backend`), not from `C:\inetpub` . That folder holds the uploads and logs, so leave the project folder in place after deployment — do not delete it.

Part I — Create the First Login User

The database starts empty (no users). There are two ways to create the first account.

Option 1 — Sign up from the website (easiest)

1. Open `http://localhost/` on the VM.
2. Go to the **Sign up** page and register with an email and password.
3. This creates the account with the **viewer** role.

Option 2 — Create an admin user directly in the database

To create an account (or promote one) with the **super_admin** role, use SQL. First, generate a password hash:

```
cd C:\Apps\flowaccel\backend
node -e "require('bcrypt').hash('YourPassword123', 12).then(h=>console.log(h))"
```

Copy the printed hash. Then create a file `C:\Temp\new-user.sql` with this content (replace the email, hash, and name):

```
WITH new_user AS (
  INSERT INTO users (email, password_hash, full_name)
  VALUES ('admin@example.com', 'PASTE_THE_HASH_HERE', 'Administrator')
  RETURNING id
), new_profile AS (
  INSERT INTO profiles (user_id, full_name, department, role, org_id, preferences)
  SELECT id, 'Administrator', '', 'super_admin',
         '971589dd-afcb-4a12-8900-47626e4d59cc',
         '{"theme":"dark","language":"en"}'
  FROM new_user
)
INSERT INTO org_members (org_id, user_id, role)
SELECT '971589dd-afcb-4a12-8900-47626e4d59cc', id, 'super_admin' FROM new_user;
```

Apply it:

```
& "C:\Program Files\PostgreSQL\15\bin\psql.exe" -U postgres -h localhost -d jotflow -f C:\Temp\new-user.sql
```

Roles available: **viewer** , **director** , **super_admin** . To change an existing user's role later:

```
UPDATE profiles SET role='super_admin' WHERE user_id=(SELECT id FROM users WHERE email='someone@example.com');
```

Part J — Verify Everything Works

Run these checks in PowerShell on the VM.

#	Command	Expected
1	<code>(Invoke-WebRequest http://localhost/ -UseBasicParsing).StatusCode</code>	<code>200</code> — IIS serves the frontend
2	<code>(Invoke-WebRequest http://localhost/api/health -UseBasicParsing).Content</code>	<code>{"ok":true,...}</code> — reverse proxy reaches the backend
3	<code>pm2 list</code>	<code>jotflow-backend</code> = <code>online</code>
4	Open <code>http://localhost/</code> in a browser and log in	Dashboard loads

If check 1 passes but check 2 returns a **500** or IIS error, the reverse proxy / ARR is not working — see Troubleshooting.

Final test — reboot the VM and confirm the site still works (after doing Part K1 for auto-start). This proves the install survives a restart.

Part K — Optional Hardening

K1 Auto-start the backend on boot

By default, after a VM reboot IIS restarts automatically but the PM2 backend does **not**. To fix that, install the PM2 startup helper (run PowerShell as Administrator):

```
npm install -g pm2-windows-startup
pm2-startup install
pm2 save
```

Now the backend is restored automatically on every boot. (Run `pm2 save` again whenever you add or change PM2 processes.)

K2 Allow access from other computers (firewall)

To let other machines on the network open the site, allow port 80 through the Windows Firewall (PowerShell as Administrator):

```
New-NetFirewallRule -DisplayName "FlowAccel HTTP" -Direction Inbound `
-Protocol TCP -LocalPort 80 -Action Allow
```

Other users then browse to `http://<VM-IP>/`. Find the IP with `ipconfig`.

K3 Enable HTTPS (recommended for production)

For a quick self-signed certificate (testing only):

```
$cert = New-SelfSignedCertificate -DnsName "flowaccel.local" `
-CertStoreLocation "cert:\LocalMachine\My"
New-WebBinding -Name "FlowAccel" -Protocol https -Port 443
(Get-WebBinding -Name "FlowAccel" -Protocol https).AddSslCertificate(
$cert.Thumbprint, "My")
```

For real production use a proper certificate (internal CA or a public CA) instead of a self-signed one, and also allow port 443 in the firewall (K2, change `-LocalPort 80` to `443`). If you serve over HTTPS, also add the `https://` address to `ALLOWED_ORIGIN` in `backend\.env` and restart the backend.

K4 Microsoft (Azure AD) single sign-on

Optional. To enable the "Sign in with Microsoft" button, fill in the `MICROSOFT_*` values in `backend\.env` with your Azure AD app registration details, and set the redirect URI in Azure to `http://<VM>/api/auth/microsoft/callback`. Restart the backend afterwards. Email/password login works fine without this.

Troubleshooting

The page shows "HTTP Error 500.19"

IIS cannot read `web.config`, usually because a locked configuration section is being overridden. The shipped `web.config` is already adjusted for this. If you edited it and re-introduced a `<websocket>` element, remove it — that section is locked at the server level. Make sure URL Rewrite and ARR (Part A3) are installed.

The site loads but login fails / `/api` calls return 502 or 404

- **Backend not running** — run `pm2 list`; if missing, `pm2 start ecosystem.config.js` (Part H).
- **ARR proxy not enabled** — re-run `deploy-to-iis.ps1`; it enables the server-level proxy. Or manually: `appcmd set config -section:system.webServer/proxy /enabled:true /commit:apphost`.
- **Port mismatch** — `web.config` proxies to `localhost:3000`; the backend's `PORT` (in `.env` and `ecosystem.config.js`) must also be `3000`.

"Your email is not a member of the JotForm workspace"

`NODE_ENV` is set to `production` but the VM cannot reach JotForm. Set `NODE_ENV=development` in `backend\.env` and run `pm2 restart jotflow-backend`.

"Missing required env var: DATABASE_URL"

The `backend\.env` file is missing, misnamed (e.g. `.env.txt`), or in the wrong folder. It must be exactly `C:\Apps\flowaccel\backend\.env`.

Database connection refused

- Check PostgreSQL is running: `Get-Service postgresql*`.
- Check the password and database name in `DATABASE_URL`.
- Confirm the database exists (Part D1) and the migration ran (D2).

Port 80 already in use

Another site (often "Default Web Site") owns port 80. The deploy script stops it automatically. To check what is on port 80: `Get-NetTCPConnection -LocalPort 80 -State Listen`.

Port 3000 already in use

Pick a free port: change `PORT` in `backend\.env` and in `ecosystem.config.js`, change the three `localhost:3000` entries in `web.config` to match, then re-run the deploy script and `pm2 restart jotflow-backend`.

Quick Reference

Folder layout after install

```
C:\Apps\flowaccel\      ← project source (keep this!)
├─ backend\            ← Express API + .env + node_modules
│  └─ .env              ← your configuration
│     └─ ecosystem.config.js ← PM2 config
├─ frontend\           ← React source
├─ dist\               ← built frontend (created by "npm run build")
├─ web.config           ← IIS reverse-proxy rules
└─ deploy-to-iis.ps1    ← deployment script

C:\inetpub\flowaccel\  ← what IIS actually serves (copy of dist + web.config)
```

Ports

Port	Used by
80	IIS — the FlowAccel website (what users open)
3000	Node backend (internal — reached via the proxy only)
5432	PostgreSQL database (internal)

Updating the app later

```
# 1. Get the new code (git pull, or copy new files)
# 2. Rebuild the frontend
cd C:\Apps\flowaccel\frontend
npm run build
# 3. Re-deploy to IIS (PowerShell as Administrator)
cd C:\Apps\flowaccel
PowerShell -ExecutionPolicy Bypass -File .\deploy-to-iis.ps1
# 4. Restart the backend
pm2 restart jotflow-backend
```

Install order — at a glance

1. Install Node.js, PostgreSQL, IIS, URL Rewrite, ARR, PM2 (Part A)
2. Copy the project to `C:\Apps\flowaccel` (Part B)
3. `npm install` in `backend` and `frontend` (Part C)
4. Create the database (Part D1)
5. Create `backend\.env` (Part E)
6. Run the migration: `node db/migrate.js` (Part D2)
7. Build the frontend: `npm run build` (Part F)
8. Deploy to IIS: `deploy-to-iis.ps1` (Part G)
9. Start the backend: `pm2 start ecosystem.config.js` (Part H)
10. Create the first user (Part I)
11. Verify (Part J)
12. Auto-start on boot (Part K1)